InboxRL: Pure Reinforcement Learning (GRPO) for Calendar Event Extraction from Institute Emails

Hemant Patel (B23CH1018)
IIT Jodhpur
b23ch1018@iitj.ac.in

Kartik Gehlot (B23EE1088)
IIT Jodhpur
b23ee1088@iitj.ac.in

Code: https://github.com/hhemantpatel/inboxrl

***Abstract*—Deadlines, quizzes and talks reach students as free-form institute emails. We train a small language model (Qwen2.5-0.5B-Instruct with LoRA) to turn such an email into a JSON list of calendar events *purely with reinforcement learning*: no supervised fine-tuning, and labels are used only inside a verifiable reward. Generation is cast as a token-level MDP and optimised with Group Relative Policy Optimization (GRPO). The reward combines a format gate with a Hungarian bipartite match between predicted and gold events, and we report two reward-hacking exploits we found in our first reward design and how we closed them. On a template-disjoint synthetic test set and on held-out anonymised real emails, [TODO: one-sentence result from results_table.tex].**

I. Problem

A student receives many emails per day; missing one deadline is costly. Extraction must resolve relative dates ("next Friday" relative to the received date), ignore superseded dates ("postponed from 3 Nov"), handle multiple events and noisy forwarded or Hinglish text, and emit machine-readable output that can be exported to an `.ics` calendar. A 0.5B model runs locally (privacy) but, prompted zero-shot, often breaks the schema or mis-resolves dates. The course rules forbid supervised methods, so we ask: *can RL with a verifiable reward alone teach this behaviour?*

II. RL Formulation

State. $s_t = (x_{\text{sys}}, x_{\text{email}}, d_0, y_{<t})$: system instructions, sender, subject and body (truncated so the prompt is $\leq$640 tokens), the received date d_0, and the tokens generated so far. The policy embeds s_t as a sequence of 896-dimensional hidden states (context $\leq 640 + 192$ tokens). **Action.** $a_t \in \mathcal{V}$, the next token; $|\mathcal{V}| = 151{,}936$. **Transition.** Deterministic, $s_{t+1} = s_t \circ a_t$; the episode ends at `<|im_end|>` or after $T = 192$ tokens. **Discount** $\gamma = 1$; the reward is sparse and terminal ($r_t = 0$ for $t < T$). **Policy** π_θ: Qwen2.5-0.5B-Instruct [1] with LoRA (r=16, α=32, all projections) [2]; the frozen base model is π_{ref}.

A. Reward

Let P be the parsed predicted events and G the gold events. If the output is not a JSON list that validates against the schema, $r_{\text{fmt}} = -1$ and no match credit is given; otherwise $r_{\text{fmt}} = +1$. For a pair (p, g) with $\delta = \mathbb{I}[p.\text{date} = g.\text{date}]$:

$$S(p,g) = 0.35\,\delta + \big(\delta + 0.25(1-\delta)\big)\big(0.20\,s_{\text{time}} + 0.25\,F_1^{\text{title}} + 0.10\,s_{\text{type}} + 0.10\,s_{\text{loc}}\big) \tag{1}$$

where F_1^{title} is bag-of-tokens F1 and s_{loc} a fuzzy string ratio. The Hungarian algorithm [3] finds the assignment M maximising total similarity, and

$$r_{\text{match}} = \frac{\sum_{(p,g)\in M} S(p,g)}{\max(|P|, |G|)}, \tag{2}$$

so both missed and hallucinated events cost reward. r_{empty} is $+1$ for `[]` on a no-event email, -1 for hallucinating on one, and -0.5 for answering `[]` when events exist. $r_{\text{len}} = -1$ if any text surrounds the JSON (-0.5 more beyond 800 characters). The total is

$$R = 0.5\,r_{\text{fmt}} + 2\,r_{\text{match}} + r_{\text{empty}} + 0.25\,r_{\text{len}} \in [-1.75,\ 2.5]. \tag{3}$$

Reward hacking found and fixed. Probing our first reward with hand-written policies exposed two exploits. (i) Titles were scored with `token_set_ratio` plus a substring bonus, so the one-letter title "a" earned 2.45 of the maximum 2.5. (ii) The date was just one additive field, so an event on the *wrong day* kept 1.8/2.5. With token-F1 titles and date gating (Eq. 1) these fall to 2.0 and 0.83, while a wrong-date answer still beats giving up (`[]` scores 0), which keeps early learning signal. Unit tests pin these values and check that constant-`[]` and "copy every date" policies score below an honest policy.

III. Environment and Data

Synthetic generator (our own code). Templates produce emails together with exact labels at five levels used as a curriculum: L1 explicit dates; L2 varied formats (05/11/2026, Nov 5th, 5 pm, EOD, noon); L3 relative dates; L4 postponements, quoted reply chains and 2–4-event digests; L5 forwarded, noisy and partly Hinglish emails ("kal shaam 5 baje LHC-101 mein quiz hai"). 15% of emails contain no event. A quarter of all templates, plus two courses and two venues, appear *only* in the test split, so the 300 synthetic test emails measure generalisation to unseen phrasings rather than template memorisation (train 2000, val 200). **Real emails.** [TODO: N] institute emails from a Google Takeout export were anonymised (roll numbers, phones, personal addresses, names) and hand-labelled; 60 form a held-out test set and the rest are mixed into the second curriculum stage at 10%. `EmailExtractionEnv` implements the MDP with `reset`/`step`.

pending: figures/comparison˙real˙test.pdf

pending: figures/training˙diagnostics.pdf

Fig. 1. Left: baselines vs. InboxRL on real emails. Right: KL, completion length and fraction of zero-variance groups during training.

TABLE I
TEST RESULTS (%). GRPO: MEAN±STD OVER 3 SEEDS.

(pending: run `python -m inboxrl.plots`*)*

pending: figures/reward˙vs˙step.pdf

Fig. 2. Training reward of GRPO rollouts and validation reward.

IV. ALGORITHM

GRPO [4] drops PPO's value network, which matters on a 4 GB GPU. For each prompt x we sample a group of $G{=}4$ completions, and the advantage of completion i is $\hat{A}_i = (R_i - \text{mean}_j\, R_j)/(\text{std}_j\, R_j + \epsilon)$, shared by all its tokens. With $\rho_{i,t} = \pi_\theta(y_{i,t}|x, y_{i,<t})/\pi_{\theta_{\text{old}}}(y_{i,t}|x, y_{i,<t})$ we maximise the token-normalised clipped objective (TRL [5], `loss_type=dapo`)

$$\mathcal{J}(\theta) = \frac{1}{\sum_i |y_i|} \sum_i \sum_t \left[\min\left(\rho_{i,t}\hat{A}_i, \text{clip}(\rho_{i,t}, 1{\pm}0.2)\hat{A}_i\right) - \beta\, \mathbb{D}_{\text{KL}}^{(t)} \right] \tag{4}$$

with $\beta{=}0.04$ against π_{ref}. Each step uses 4 prompts $\times$ 4 completions, temperature 0.9, learning rate $2{\times}10^{-5}$ (cosine, 50 warm-up steps), [TODO: N] steps. **Curriculum**: the first 250 steps draw only L1–L2 emails, after which all levels are mixed.

V. EXPERIMENTS

Baselines: constant [] and random-guess policies (reward sanity checks), a regex+`dateparser` rule system, and zero-/3-shot prompting of the same 0.5B model and of Qwen2.5-1.5B. No SFT baseline is included because the assignment forbids supervised training. Event F1 counts a predicted event as correct when its date is right and its title token-F1 is $\geq$0.5; brackets in the plots are 95% bootstrap confidence intervals over emails.

Results. [TODO: Describe Table I honestly: does GRPO beat zero-shot / few-shot / the 1.5B model, by how much, and are the CIs separated? Mention per-level F1 (figures/levels_*.pdf).]

Ablations. [TODO: Summarise Table II: curriculum, KL, format-only reward, group size.]

Failure analysis. [TODO: 2–3 real failure cases from results/completions/*.jsonl.]

VI. CONCLUSION

[TODO: Two sentences from the final results.] The trained adapter ships with a command-line tool that turns an mbox export into an `.ics` file for Google Calendar.

TABLE II
ABLATIONS ON THE SYNTHETIC TEST SET.

(pending: run `python -m inboxrl.plots`*)*

REFERENCES

[1] Q. Team, "Qwen2.5: A party of foundation models," *arXiv preprint arXiv:2412.15115*, 2024.

[2] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "Lora: Low-rank adaptation of large language models," *arXiv preprint arXiv:2106.09685*, 2021.

[3] H. W. Kuhn, "The hungarian method for the assignment problem," *Naval research logistics quarterly*, vol. 2, no. 1-2, pp. 83–97, 1955.

[4] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, M. Zhang, Y. Zhang, Y. Wu, and D. Guo, "Deepseekmath: Pushing the limits of mathematical reasoning in open language models," *arXiv preprint arXiv:2402.03300*, 2024.

[5] L. von Werra, Y. Belkada, L. Tunstall, E. Beeching, T. Thrush, N. Lambert, and S. Huang, "Trl: Transformer reinforcement learning," https://github.com/huggingface/trl, 2022.